bwtrack: Real-Time Per-Process Network Bandwidth Tracking with eBPF and Its Accuracy under Packet Loss

CS331 Computer Networks · Project T001 (Topic 4) · IIT Gandhinagar · Instructor: Prof. Sameer Kulkarni
Teaching Assistants: Jainish, Shubham

Kshitij Kasodkar
23110175

Hanamanthagouda Policepatil
23110126

Manav Mangal Jain
22110140

Thipparapu Rushitha
23110338

Ralebhat Priyanka Shriram
24110278

I. Introduction

Knowing *which program* uses the network is a basic operational question: a saturated uplink, an unexpected data bill, or a background updater stalling a video call all start with "who is sending?". Linux readily reports traffic per network interface (`/proc/net/dev`) and tools such as tcpdump show every packet, but neither attributes bytes to processes: by the time a packet reaches the network card, the kernel no longer tracks which process created it.

This project builds **bwtrack**, a real-time per-process bandwidth tracker based on **eBPF** (extended Berkeley Packet Filter), which lets verified programs run inside the running Linux kernel. We attach probes at the *socket layer*, where the kernel executes in the context of the application that moves the data, count bytes per process and remote endpoint in kernel maps, and expose them through a terminal UI, a web dashboard, a SQLite history and a Prometheus endpoint. Following the mid-semester review, we study what "accuracy" means for real-time tracking and back every claim with a test that compares the tracker against ground truth printed by the workload itself, run on a Raspberry Pi 5.

Contributions:

- An eBPF tracker that attributes TCP and UDP traffic (IPv4/IPv6, including unconnected UDP) to processes and remote endpoints, with correct handling of partial sends, cumulative receive callbacks, end-of-stream FIN, nested IPv6/IPv4 UDP calls, short-lived connections and socket-address reuse.
- A three-layer measurement view (application/socket, TCP wire, NIC) with live TCP retransmission and UDP receive-buffer drop events attributed to processes.
- A loss experiment with independent ground truth (byte counts and UDP sequence numbers at both ends) and `tc netem` loss in both directions, repeated three times plus a 20 ms delay run on a Raspberry Pi 5 over Wi-Fi.
- A proof suite of 17 automated tests for correctness, performance, scalability and reliability. It found two bugs in our own tracker, which we fixed.
- A portable implementation (kernels 5.15–6.18, x86-64 and ARM64) including automatic repair of a BCC/kernel-header mismatch that otherwise prevents compilation on the Pi's 6.18 kernel.

II. Problem Statement

Problem. Given a Linux host, continuously report for every process *p* and remote endpoint *e* (IP, port, protocol) the number of bytes *p* sent to and received from *e*, the current rates, and, for TCP, the bytes actually transmitted including retransmissions; keep a queryable history; and quantify how close these real-time numbers are to the truth under packet loss, at an overhead users do not notice.

Inputs: live kernel events (socket send/receive calls, TCP retransmissions, state changes, UDP enqueue failures) and interface counters. **Outputs:** per-process and per-flow rates and totals, TCP wire statistics (wire bytes, retransmitted bytes and segments, RTT, congestion window), loss events, history and reports.

Assumptions and constraints: root privileges; a kernel with kprobes and BPF events (≥ 5.5); kernel headers available (the Raspberry Pi OS kernel has no BTF, so CO-RE is not an option); no kernel modification; no Python packages beyond the distribution's (Raspberry Pi OS blocks system-wide `pip`).

Scope. In scope: TCP and UDP over IPv4/IPv6 from user processes, live and historical views, accuracy under loss and delay, overhead, scalability and stability. Out of scope: ICMP and raw sockets, kernel-originated traffic, per-container labelling, decrypting TLS, and traffic of Windows applications when running under WSL2 (a separate VM).

Gap in existing solutions. Packet capture (tcpdump, iftop) has no process information; *nethogs* captures packets and *guesses* the owner by matching ports to `/proc` socket tables, which is costly and misses short-lived flows; polling `ss`/`sock_diag` misses connections that open and close between polls and has no UDP byte

counters; firewall/cgroup accounting is per user or per group. Our mid-semester prototype ran only with mock data and claimed 100 % accuracy from a loopback test, which cannot lose packets. None of these explain *why* real-time numbers differ between layers.

III. Project Objectives

- O1. Attribute every TCP/UDP byte sent or received by a user process to its PID and remote endpoint; application-layer totals must equal ground truth (verified by 9 correctness tests).
- O2. Run live on real hardware (Raspberry Pi 5, ARM64, kernel 6.18) and on x86-64 WSL2 (kernel 5.15), with no mock data.
- O3. Expose TCP wire bytes, retransmissions, RTT and UDP receive drops per process, and detect loss events live.
- O4. Quantify real-time accuracy at three layers under 0–10 % loss, with and without 20 ms added delay, over three repeats.
- O5. Keep overhead unnoticeable on a real network; measure it with and without the tracker.
- O6. Scale to hundreds of simultaneous connections and stay stable for 30 minutes with thousands of short-lived processes.
- O7. Store history at ≤ 10 s resolution with reports by program, remote IP, protocol and time.

IV. Architecture and Design

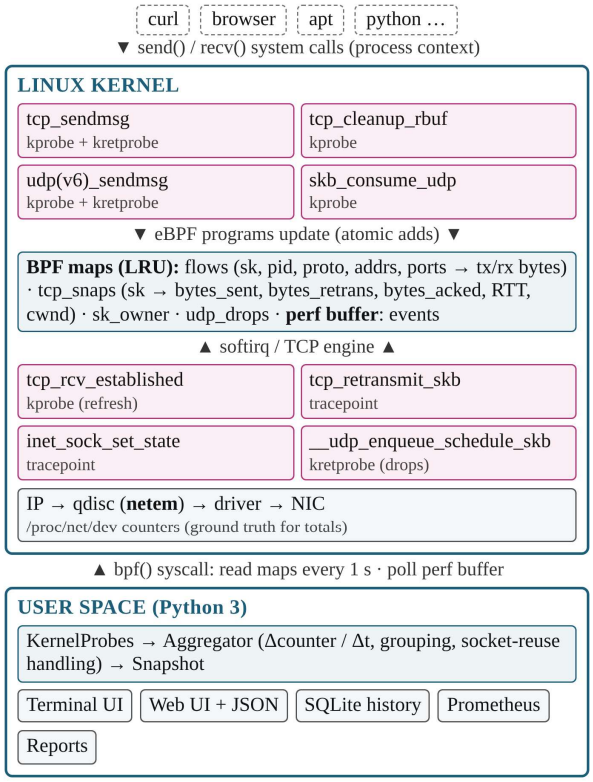


Fig. 1. System architecture. Socket-layer probes run in process context and update LRU BPF maps; softirq-side probes add TCP wire statistics and loss events; user space reads the maps every 0.5-1 s.

Kernel side. eBPF programs written in restricted C are compiled by BCC against the running kernel's headers, checked by the verifier, JIT-compiled and attached (Table I). Socket-layer probes execute in the application's context, so `bpf_get_current_pid_tgid()` identifies the true owner. They read the socket's 5-tuple from `struct sock` and add the byte count atomically to the `flows` map entry keyed by (socket, TGID, family, protocol, local/remote address and port). TCP's RFC 4898 counters are copied from `struct tcp_sock` into `tcp_snaps` (rate-limited to 100 ms per socket). Softirq-side hooks never trust the current PID; they look the owner up in `sk_owner`. Rare events (retransmissions, drops) are streamed through a perf buffer.

Connection lifecycle. The `inet_sock_set_state` tracepoint starts every new connection's wire counters at zero (SYN), so the first bytes of a connection are counted. When a connection reaches `TCP_CLOSE`, its final counters are added to `closed_*` fields of its flow entry and the snapshot is deleted. User space then counts *live snapshot + closed total* per flow, and subtracts what it had already counted from the live snapshot. A connection that opens and closes between two reads therefore still reports its wire bytes. If the close event is

missed (a tracepoint program is skipped when another eBPF program is running on the same CPU), the old socket is retired with its last snapshot when the address is reused.

TABLE I
KERNEL HOOKS

Hook	Type	Purpose
`tcp_sendmsg`	kprobe + kretprobe	TCP bytes sent; return value = bytes accepted
`tcp_cleanup_rbuf`	kprobe	TCP bytes read; advance of `copied_seq` (FIN excluded)
`udp(v6)_sendmsg`	kprobe + kretprobe	UDP sent; destination from `msg_name`
`skb_consume_udp`	kprobe	UDP received (v4+v6); peer from skb headers
`tcp_rcv_established`	kprobe	refresh wire counters while app blocks
`tcp_retransmit_skb`	tracepoint	every TCP retransmission → event
`inet_sock_set_state`	tracepoint	zero baseline at SYN; fold final counters at close
`__udp_enqueue_schedule_skb`	kretprobe	UDP receive-buffer drops

User space. `KernelProbes` loads and attaches the programs and reads the maps; `Aggregator` turns cumulative counters into per-interval deltas and rates, joins TCP statistics to flows, groups rows per process and remote IP, expires idle sockets while keeping per-process totals, and handles socket-address reuse. A collector thread produces one *Snapshot* per interval and fans it out to the terminal UI, web server, SQLite recorder and Prometheus exporter. Rates use a monotonic clock.

Three measurement layers. The *application/socket* layer (our kprobes) is exact per process; the *TCP wire* layer adds retransmissions and reflects when TCP actually transmits; the *NIC* layer (`/proc/net/dev`) adds all headers and every packet on the interface but has no process. Reporting all three makes the differences caused by buffering, loss and headers visible.

A. Design decisions and alternatives

TABLE II
DESIGN CHOICES

Chosen	Alternative	Reason / trade-off
Socket-layer kprobes	XDP/TC, libpcap, nethogs	Only place with process context; misses headers (added via TCP/NIC layers)
kretprobe return value	`size` argument	Accepted vs requested bytes; needs entry/return map
In-kernel aggregation, periodic reads	perf event per send	Constant cost, no loss when user space lags
LRU hash maps	plain hash	Bounded memory; may evict very old idle flows
TCP's own counters	counting segments	Exact, maintained by TCP; refresh is rate-limited
BCC	libbpf + CO-RE	Pi kernel lacks BTF; costs LLVM on device (220 MB RSS) and 5–9 s start
Python stdlib, SQLite	Flask, InfluxDB	Installable on Pi OS; simple; single-host scale

V. IMPLEMENTATION

Stack. Kernel side: ~700 lines of restricted C compiled by BCC 0.29–0.31 (clang/LLVM). User side: ~2,700 lines of Python 3.10+ using only the standard library (`curses`, `http.server`, `sqlite3`) plus the distribution's `python3-bpfcc`. Web UI: one HTML page with vanilla JavaScript. Hardware: Raspberry Pi 5 (8 GB). Experiments: `tc`/netem with IFB, our traffic generator `trafficgen.py` (also runs on Windows), iperf3, and the proof suite (`proof_suite.py` with `proof/worker.py`).

Send path (pseudocode):

```
on kprobe tcp_sendmsg(sk, msg, size):
    args[pid_tgid] = sk
on kretprobe tcp_sendmsg -> ret:
    sk = args.pop(pid_tgid); if ret <= 0: return
    refresh tcp_snaps[sk] (<= every 100 ms)
    key = (sk, tgid, TCP, addrs/ports from sk)
    atomic_add(flows[key].tx_bytes, ret)
    sk_owner[sk] = (tgid, comm, key)
on tracepoint inet_sock_set_state(sk, new):
    SYN_SENT/SYN_RECV: tcp_snaps[sk] = zeros
    CLOSE: flows[sk_owner[sk].key].closed +=
final
          delete tcp_snaps[sk], sk_owner[sk]
```

Aggregation (every interval Δt): for each flow, Δ = counter - previous (or counter if it went backwards, i.e. the entry was recreated); rate = Δ/Δt. Live TCP deltas are attached to the most recently active flow used after the socket's birth; closed totals are added to their own flow minus what was already counted live.

A. Challenges and solutions

- **BCC vs kernel 6.18 on the Pi:** BCC bundles an older `uapi/linux/bpf.h`, so compilation failed on

`struct bpf_task_work`, `BPF_LOAD_ACQ`, `BPF_STORE_REL`. `compat.py` parses clang's errors, copies the missing definitions from the running kernel's own uapi header, retries and caches the result (3 definitions; 9.3 s first compile, 5.0 s afterwards).

- **Bugs found by the proof suite:** (1) connections that opened and closed between two reads reported 0 wire bytes, and the first chunk of every connection was missing from the wire count (19 B short in the 26 Oct run); fixed by the lifecycle handling above. (2) The peer's FIN advances `copied_seq` by one, so downloads were counted 1 byte too high; we exclude it when the stream end has been consumed.
- **Receive double counting:** `tcp_cleanup_rbuf` may be called repeatedly with a cumulative `copied`; we count the advance of `tp->copied_seq`.
- **Unconnected UDP and IPv6 nesting:** destination read from `msg->msg_name`; `udpv6_sendmsg` → `udp_sendmsg` nesting counted once.
- **Loopback:** remote addresses that are loopback or any host-owned address are marked and hidden by default to avoid double counting.
- **Honest overhead measurement:** without the tracker, loopback TCP on the Pi alternated between 26 and 70 Gbit/s depending on core placement; we pin iperf3 to cores 2 and 3 and measure OFF → ON → OFF.

Code repository:
https://github.com/KshitijKasodkar/CS331_T001_PerProcess_eBPF_Final

VI. Testbed Setup

TABLE III
Testbed

Component	Configuration
Device under test	Raspberry Pi 5 Model B, 8 GB, microSD
OS / kernel	Raspberry Pi OS Lite 64-bit, 6.18.50+rpt-rpi-2712, aarch64, Python 3.13.5
eBPF	BCC 0.31.0, linux-headers-rpi-2712
Link	Wi-Fi `wlan0` via phone hotspot (Pi 10.58.105.107, laptop 10.58.105.150)
Peer	Windows 11 laptop, Python 3.13, `trafficgen.py server`
Loss / delay	`tc netem loss p%` on egress + IFB ingress; p ∈ {0,1,2,5,10}; one run with +20 ms each way
Workload / level	TCP up & down 20 MiB; UDP up & down 20 Mbit/s for 5 s, 1200 B datagrams
Repeats	3 loss runs (30–31 Oct 2026, 23:52–00:07); mean ± sd
Proof suite	17 tests on the Pi (23:15–23:52), 30-min soak; follow-ups at 00:30 and 00:50
Also run on	WSL2 5.15 (BCC 0.29)

Ground truth is independent of eBPF: the traffic generator counts bytes at both ends, and each UDP datagram carries a session id and sequence number, so true loss = 1 - received/sent. In the proof suite, every workload process prints its own PID and the exact bytes it sent or received, and iperf3 reports its own byte counts. NIC counters are read before and after each transfer.

VII. Evaluation

Metrics. *App accuracy* = eBPF socket-layer bytes ÷ application count; *wire extra* = (Δ`bytes_sent` - app bytes) ÷ app bytes; *retx %* = retransmitted ÷ wire bytes; *per-sample deviation* = |app - wire| ÷ wire over 0.5 s samples; *wire lag* = time between the last sample with app bytes and the last with wire bytes; *goodput* = app bytes × 8 ÷ duration.

A. Accuracy under packet loss

TABLE IV
TCP upload, mean ± sd of 3 runs (Pi 5, Wi-Fi)

Loss	App acc.	Wire extra %	Retx %	Retx segs	Dev. %	Lag s	Mbit/s
0 %	100.0	0.89 ± 1.54	0.86	128	29	0.2	178 ± 15
1 %	100.0	1.24 ± 0.76	1.22	179	41	0.2	156 ± 29
2 %	100.0	1.72 ± 0.32	1.69	250	41	0.3	132 ± 36
5 %	100.0	6.30 ± 0.45	5.92	912	30	1.5	12.5 ± 3.2
10 %*	100.0	12.07 ± 0.05	10.77	1749	90	17.8	1.5 ± 0.5

*n = 2 at 10 %: in run 3 our traffic generator timed out waiting for the final reply while TCP was backing off; the timeout has since been raised.

Application-layer totals are exact. In all 59 completed loss trials (3 runs × 5 levels × 4 modes; one of the 60 timed out, see above) and in the receive-buffer test the socket-layer count equals the bytes the application sent or received. TCP hides loss from applications by retransmitting, so loss changes the wire, not the application stream. With the lifecycle fix, wire bytes equal app bytes exactly when nothing is lost (+0.000 % in runs 2 and 3; run 1 had 274 real Wi-Fi retransmissions even at 0 % netem loss).

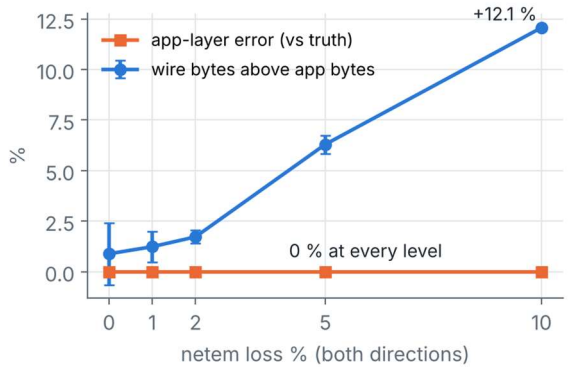


Fig. 2. TCP upload: extra wire bytes vs configured loss (mean ± sd, 3 runs); the application-layer error is 0 at every level.

Real-time numbers are not exact. Per 0.5 s sample, app and wire bytes differ by 29-90 % on average: `send()` returns as soon as data is in the socket buffer, while TCP transmits it later and again after losses. At 10 % loss the application finished 9-27 s before TCP had delivered everything (Fig. 3). Totals converge; instants do not.

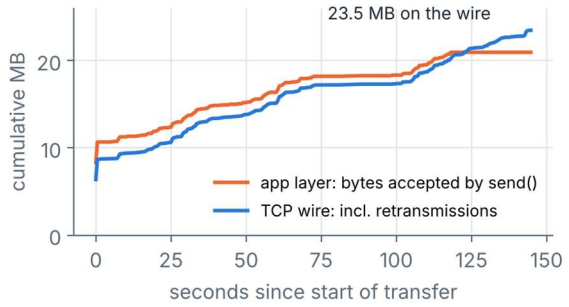


Fig. 3. Cumulative bytes during the 10 % loss upload (run 1): the app stops at 21.0 MB, the wire carries on to 23.5 MB for another 26.5 s.

TABLE V
UDP: SENDER VIEW VS DELIVERED (MEAN ± SD, 3 RUNS)

Loss	Upload: sender eBPF	Upload: true loss	Download: true loss	Download: eBPF loss
0 %	100 % sent	0.05 ± 0.09 %	0.22 ± 0.23 %	0.22 ± 0.23 %
1 %	100 % sent	0.99 ± 0.12 %	1.04 ± 0.05 %	1.04 ± 0.05 %
2 %	100 % sent	2.06 ± 0.43 %	2.07 ± 0.08 %	2.07 ± 0.08 %
5 %	100 % sent	5.23 ± 0.35 %	5.10 ± 0.24 %	5.10 ± 0.24 %
10 %	100 % sent	9.69 ± 0.38 %	10.15 ± 0.17 %	10.15 ± 0.17 %

UDP loss is invisible to the sender. The sender's counters always report 100 % sent; only the receiver can observe loss. On the receiving side (Pi downloads), eBPF-measured loss equals the sequence-number ground truth in all 15 trials (Fig. 4). In the receive-buffer test the kernel's `RcvbufErrors` increased by 2985 and eBPF counted 2985 drops.

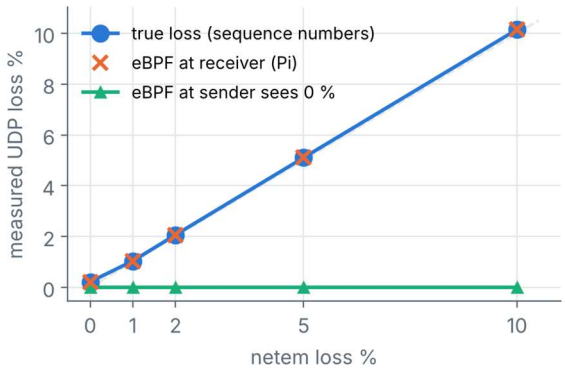


Fig. 4. UDP: true loss, eBPF loss at the receiver, and what the sender's eBPF counters report.

B. Effect of delay: comparison with TCP theory

With 20 ms added each way, 1 % loss cuts upload goodput from 156 to 3.7 Mbit/s (Fig. 5). Using the RTT our tracker measured, the Mathis bound $(MSS/RTT)\cdot(1.22/\sqrt{p})$ with MSS = 1448 B predicts 2.8, 2.0, 1.1 and 0.57 Mbit/s at 1, 2, 5 and 10 % loss; we measured 3.7, 2.5, 1.4 and 0.46 Mbit/s. Without added delay, the short Wi-Fi RTT and CUBIC with SACK/RACK keep 20 MiB transfers far above the bound at 1–2 % loss, and they collapse only at 5–10 %. App-layer accuracy stayed 100 % in all delay trials.

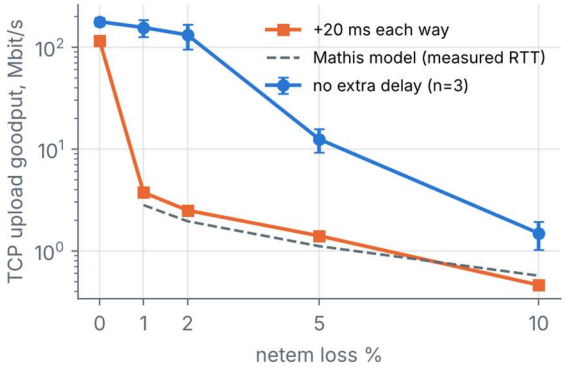


Fig. 5. TCP upload goodput vs loss without and with +20 ms delay, and the Mathis bound computed from the measured RTT (log scale).

C. Correctness tests (proof suite)

TABLE VI
CORRECTNESS ON THE PI: TRACKER VS. BYTES PRINTED BY THE WORKLOAD

Test	Result
Non-blocking sends (164 partial, 165 EAGAIN)	15,535,328 B = accepted; requested size would give +108 %
`MSG_WAITALL` receive	20,971,520 B = app
`sendfile()`	20,971,520 B = app
Unconnected UDP to 3 IPs	300,000 / 600,000 / 900,000 B exact; 0 B to 0.0.0.0
IPv6 TCP, UDP, v4-mapped UDP	10,485,760 / 500,000 / 500,000 B exact
8 threads in one process	41,943,040 B under one PID
`fork()`: parent + 4 children	4/4 children and parent exact, own PIDs
500 short connections	app and wire 10,000,000 B exact
Loopback and own LAN IP	both marked as local traffic

D. Non-functional parameters

TABLE VII
THROUGHPUT WITH VS WITHOUT THE TRACKER (OFF-ON-OFF, 4 RUNS PER BLOCK)

Workload	Without	With	Change
Wi-Fi TCP upload (Mbit/s)	174.6 ± 8.5	176.2 ± 9.1	+0.9 %
Wi-Fi TCP download (Mbit/s)	125.1 ± 13.3	120.7 ± 17.8	−3.5 %
Loopback TCP, pinned (Gbit/s)	69.5 ± 1.0	58.0 ± 0.9	−16.5 %
Loopback UDP 64 B (k pkt/s)	500.1 ± 2.7	248.4 ± 0.6	−50.3 %

Performance. On the real Wi-Fi path the difference is within run-to-run variation. The cost of the in-kernel probes appears only at extreme event rates: about 0.6 µs per `tcp_sendmsg` return (603.6 ns), 433 ns per `tcp_cleanup_rbuf`, 98 ns per `tcp_rcv_established`, 294 ns on average over 7.3 M runs. The tracker process used 0.29 % CPU while busy and its BPF maps 11.4 MB; its resident memory is 220 MB because BCC keeps the LLVM compiler loaded.

Scalability. With 1, 10, 50, 100 and 200 simultaneous connections to distinct IPs, every connection was attributed exactly; collector CPU rose from 4.0 to 12.4 % and a full map read from 8.7 to 23 ms. 170,388 tiny `send()` calls per second (87,238,848 B) were counted exactly. With 50 processes started at once, the suite saw 49/50 exact once; a follow-up of 5 × 50 processes, checked both early and late, gave 250/250.

Reliability. The real `bwtrack record` process ran for 30 min while 2,511 short-lived processes used TCP and UDP: all 2,511 totals in the SQLite history equal the truth; CPU 1.9 %; memory 220.3 → 223.3 MB (Fig. 6). No kretprobe misses or lost perf events occurred in any test. With the flow map forced to 256 entries and 600 flows, LRU eviction kept memory bounded and the tool kept running (with the default 16,384 entries all bytes were captured). Starting the tracker during a 2 MB/s transfer reported at most 2.03 MB/s: no spike from earlier bytes.

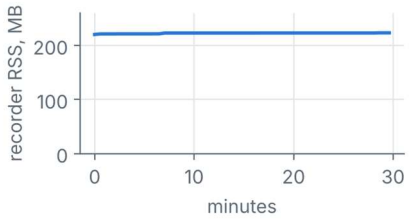


Fig. 6. Resident memory of the recorder during the 30-minute soak test.

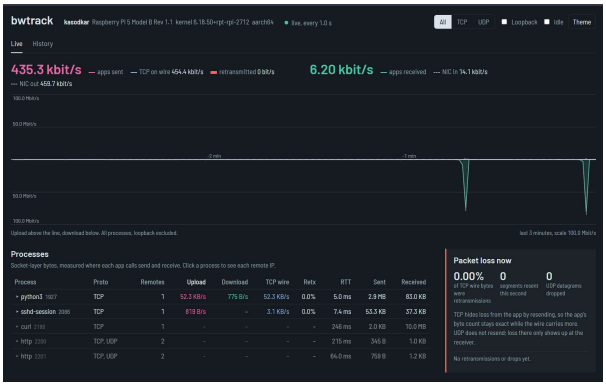


Fig. 7. Web dashboard on the Raspberry Pi: per-process rates, TCP wire, RTT and packet-loss panel.

VIII. SUMMARY

We built bwtrack, an eBPF-based tracker that attributes TCP and UDP traffic to processes and remote endpoints by probing the kernel's socket-layer send and receive paths, enriches it with TCP's own wire and retransmission counters and live loss events, and presents it through terminal, web, history and Prometheus interfaces. On a Raspberry Pi 5 over Wi-Fi with 0–10 % injected loss, application-layer totals matched ground truth in all 59 trials over three runs, while wire bytes grew with loss (+12.1 % at 10 %), per-sample rates differed from the wire by 29–90 % with up to 27 s lag, and UDP loss was invisible to senders but measured exactly at receivers. With 20 ms delay, goodput followed the Mathis model computed from our own RTT measurements. On the real network the tracker caused no measurable slowdown; it scaled to 200 simultaneous connections and stayed stable for 30 minutes. Real-time bandwidth is a layer-dependent quantity, and a tracker should report the layers rather than one "100 % accurate" number.

Limitations and future work: probe cost becomes visible for floods of tiny packets (−50 % at 500 k packets/s on loopback); 220 MB memory because of BCC/LLVM (future: libbpf + CO-RE once the Pi kernel

has BTF, or fentry/fexit); payload rather than header-inclusive bytes per process; ICMP/raw/kernel traffic not attributed; container PIDs not labelled (future: cgroup id); TLS sizes are ciphertext; evaluation on one Wi-Fi link (future: Ethernet, multiple competing flows).

IX. Reproducibility

GitHub:
https://github.com/KshitijKasodkar/CS331_T001_PerProcess_eBPF_Final. The repository contains the code, all raw Pi results (JSON/CSV/HTML) under `Code/experiments/pi_results/2026-10-30_final/`, setup scripts and documentation.

1) Flash Raspberry Pi OS Lite 64-bit (SSH + Wi-Fi enabled), copy `Code/` to the Pi and run `sudo ./scripts/setup_pi.sh` (installs BCC, headers, iproute2, iperf3; runs `bwtrack doctor --compile`).
2) On the peer laptop: `python trafficgen.py server --port 5300` (allow it through the firewall).
3) On the Pi: `sudo bash experiments/run_final.sh <laptop-ip>` (about 70 min: proof suite incl. 30-min soak, loss experiment × 3, 20 ms delay run; writes `~/final_results.zip`).
4) Overhead check: `sudo python3 experiments/proof/overhead_check.py --peer <laptop-ip>`; 50-process follow-up: `sudo python3 experiments/proof/diag_processes.py`; unit tests: `python3 -m unittest discover -s tests`.

References

[1] B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.

[2] S. McCanne and V. Jacobson, "The BSD packet filter: A new architecture for user-level packet capture," in *Proc. USENIX Winter Conf.*, San Diego, CA, USA, 1993, pp. 259–270.

[3] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 27, no. 3, pp. 67–82, Jul. 1997.

[4] M. Mathis, J. Heffner, and R. Raghunarayan, "TCP Extended Statistics MIB," IETF RFC 4898, May 2007.

[5] S. Hemminger, "Network emulation with NetEm," in *Proc. Linux Conf. Australia*, Canberra, Australia, 2005.

[6] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.

[7] IO Visor Project, "BCC — Tools for BPF-based Linux IO analysis, networking, monitoring, and more." [Online]. Available: https://github.com/iovisor/bcc

[8] The Linux Kernel Documentation, "Kernel Probes (Kprobes)" and "BPF Documentation." [Online]. Available: https://docs.kernel.org

[9] Raspberry Pi Ltd., "Raspberry Pi 5 product brief" and "Raspberry Pi OS." [Online]. Available: https://www.raspberrypi.com

APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work Distribution

- Ratings are agreed by the whole team and must be UNIQUE: no two members may receive the same rating (no ties).
- Scale: 1-10 (10 = highest contribution).

TABLE VIII
WORK DISTRIBUTION AND MEMBER RATING (AGREED BY THE TEAM)

#	Member Name	Roll No.	Tasks Done	Rating (1-10, unique)
1	Kshitij Kasodkar	23110175	eBPF kernel program (all probes and maps) and the collector/aggregation logic; kernel 6.18 compatibility fix; Raspberry Pi 5 setup; final runs on the Pi; integration	10
2	Hanamanthagouda Policepatil	23110126	Packet-loss experiment framework (traffic generator, netem/IFB, ground truth); repeated and delay runs; WSL2 setup; Architecture slide	9
3	Manav Mangal Jain	22110140	Terminal UI, web dashboard, SQLite history and reports; Evaluation slide	8
4	Thipparapu Rushitha	23110338	Proof suite tests (correctness, scalability) and result charts; Non-functional slide	7
5	Ralebhat Priyanka Shriram	24110278	Report, README and documentation; testing on laptops; Challenges slide	6

APPENDIX B: AI USAGE DETAILS

TABLE IX
AI TOOLS USED

#	AI Tool	Purpose
1	Google Antigravity CLI (Gemini)	Mid-semester prototype: scaffolding the first BCC loader, UI and tests
2	Claude (Anthropic), agentic coding session	Final version: diagnosing the mid-semester failures, rebuilding the eBPF program and collector, cross-kernel testing in VMs (kernels 5.15/6.8/7.0 headers), BCC/6.18 compatibility fix, proof suite and the two bug fixes it revealed
3	Claude (Anthropic)	Raspberry Pi setup guidance, experiment and test design (netem/IFB, ground truth, OFF-ON-OFF overhead), result interpretation
4	Claude (Anthropic)	Documentation: README, concept notes, viva study guide, presentation and this report draft (reviewed and edited by the team)

B.1 Prompts

The most significant prompts and the AI's responses are documented in the repository folder `AI_Used/` (`Prompts.md`, `Step_by_Step_Details.md`, `Thought_Process.md`, `Tools.md`). Every AI-generated component was run and verified by the team on real hardware; all reported numbers come from our own Raspberry Pi measurements.

APPENDIX C: PROOF SCREENSHOTS (RASPBERRY PI 5, 30–31 OCT 2026)

Unedited terminal output of the runs behind Section VII (cropped). The full logs and per-test JSON files are in `Code/experiments/pi_results/2026-10-30_final/`.

```
 load), 0.20 % idle; RSS 220.5 MB; accuracy at full speed 100.000 %, 100.000 %, 100.000 %

loading tracker ...
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.0s. Attaching probes...
   Attached 13 probes, skipped 2 optional

== correctness (issues fixed) ==
  PASS partial_sends: tracker 15535328 B = accepted 15535328 B; counting the requested size (old method) would report 32374784 B (+
108 %)
  PASS recv_waitall: receiver with MSG_WAITALL: tracker 20971520 B, app 20971520 B
  PASS sendfile: sendfile(): tracker 20971520 B, app 20971520 B
  PASS udp_unconnected_multi_dest: one sendto() socket -> 3 IPs: 127.0.0.2 300000/300000, 127.0.0.3 600000/600000, 127.0.0.4 900000
/900000
  PASS ipv6: TCP ::1 10485760/10485760, UDP ::1 500000/500000, UDP v4-mapped 500000/500000 (not double counted)
  PASS threads_one_pid: 8 threads -> one PID: 41943040/41943040 B, 8 sockets
  PASS fork_separate_pids: parent + 4 forked children: 4/4 children exact, parent 1048576/1048576
  PASS short_connections_reuse: 500 short connections: app 10000000/10000000 B, wire 10000000 B (100.00 %)
  PASS loopback_detection: 127.x flagged: True; own IP 10.58.105.107 flagged: True

== scalability ==
       1 connections: 1/1 exact, collector CPU 4.0 %
      10 connections: 10/10 exact, collector CPU 6.81 %
      50 connections: 50/50 exact, collector CPU 5.96 %
     100 connections: 100/100 exact, collector CPU 7.22 %
     200 connections: 200/200 exact, collector CPU 12.43 %
  PASS concurrent_connections: 1 conns: 1/1 exact; 10 conns: 10/10 exact; 50 conns: 50/50 exact; 100 conns: 100/100 exact; 200 conn
s: 200/200 exact
  FAIL concurrent_processes: 50 processes at once: 49/50 attributed exactly
  PASS event_rate: 170,388 send() calls/s of 64 B: tracker 87238848/87238848 B; send probes on_tcp_send_enter 143.1 ns, on_tcp_send
_return 603.6 ns
  ebpf_cost_all_tests: 7,338,783 eBPF runs, avg 294 ns; maps 11.4 MB; kretprobe misses 0, perf lost 0

== reliability ==
```

Fig. 8. Proof suite on the Pi: 9/9 correctness tests, scalability up to 200 connections, eBPF cost (Section VII-C, VII-D).

```
== reliability ==
  PASS map_overflow_lru: 600 UDP flows: default map 100.0 % captured; forced 256-entry map: tool alive=True, 0.6 % captured (LRU ev
icts, memory stays bounded, no crash)
  PASS start_mid_transfer: tracker started 5 s into a 2 MB/s transfer: max reported 2.03 MB/s (no spike from earlier bytes)
```

Fig. 9. Proof suite: reliability tests (map overflow, start in the middle of a transfer).

```
--- UDP receive-buffer overflow test ---
  rcvbuf   kernel drops 2985  eBPF drops 2985  rx accuracy 100.0%

Wrote experiments/results/final_20261030-2315/loss_run1/20261030-235212/report.md, report.html, results.csv, series.csv

== loss experiment run 2/3 ==
Loading eBPF tracker... (iface wlan0, peer 10.58.105.150)
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.0s. Attaching probes...
   Attached 13 probes, skipped 2 optional

--- netem loss 0.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.000%  retx 0.0% (0 segs)  goodput 185.127 Mbit/s  rtt 31.3 ms  wire lag 0.0 s
  tcp-down app accuracy 100.0%  goodput 119.653 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 0.0%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 0.0%  eBPF-measured loss 0.0%

--- netem loss 1.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.621%  retx 0.618% (90 segs)  goodput 178.824 Mbit/s  rtt 27.96 ms  wire lag 0.0 s
  tcp-down app accuracy 100.0%  goodput 16.146 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 1.123%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 1.018%  eBPF-measured loss 1.018%

--- netem loss 2.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +1.705%  retx 1.677% (247 segs)  goodput 142.216 Mbit/s  rtt 11.91 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 13.016 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 1.843%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 2.112%  eBPF-measured loss 2.112%

--- netem loss 5.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +6.090%  retx 5.74% (882 segs)  goodput 16.101 Mbit/s  rtt 7.26 ms  wire lag 1.5 s
  tcp-down app accuracy 100.0%  goodput 8.708 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 5.117%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 4.935%  eBPF-measured loss 4.935%

--- netem loss 10.0% delay 0.0 ms on wlan0 (both) ---
```

Fig. 10. Loss experiment, run 2 of 3 (0-5 %), and the UDP receive-buffer test of run 1 (Tables IV-V).

```
--- netem loss 10.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +12.111%  retx 10.802% (1755 segs)  goodput 1.812 Mbit/s  rtt 23.13 ms  wire lag 9.01 s
  tcp-down app accuracy 100.0%  goodput 6.439 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 9.254%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 10.022%  eBPF-measured loss 10.022%

Wrote experiments/results/final_20261030-2315/loss_run2/20261030-235752/report.md, report.html, results.csv, series.csv
```

Fig. 11. Run 2 at 10 % loss: app bytes exact, wire +12.1 % (1755 retransmitted segments), UDP loss at the receiver = true loss (10.022 %).

```
== loss experiment with 20 ms extra delay ==
Loading eBPF tracker... (iface wlan0, peer 10.58.105.150)
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.1s. Attaching probes...
   Attached 13 probes, skipped 2 optional

--- netem loss 0.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.000%  retx 0.0% (0 segs)  goodput 115.704 Mbit/s  rtt 70.57 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 130.939 Mbit/s

--- netem loss 1.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +1.049%  retx 1.039% (152 segs)  goodput 3.741 Mbit/s  rtt 50.26 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 3.077 Mbit/s

--- netem loss 2.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +2.196%  retx 2.148% (318 segs)  goodput 2.498 Mbit/s  rtt 51.19 ms  wire lag 1.0 s
  tcp-down app accuracy 100.0%  goodput 2.45 Mbit/s
```

Fig. 12. The +20 ms delay run: app accuracy 100 %, goodput 115.7 → 3.7 Mbit/s at 1 % loss (Section VII-B).

```
# Throughput with vs without the tracker (pinned cores, OFF-ON-OFF)

Raspberry Pi 5 Model B Rev 1.1, kernel 6.18.50+rpt-rpi-2712. 4 runs per block. Tracker process CPU while ON: 0.29 %, RSS 220.5 MB.

| workload | without (mean ± sd, n) | with (mean ± sd, n) | change of mean | change of median |
|---|---|---|---|---|
| tcp_loopback_gbps | 69.468 ± 0.998 (8) | 58.008 ± 0.939 (4) | −16.5 % | −16.4 % |
| udp64_loopback_kpps | 500.134 ± 2.706 (8) | 248.356 ± 0.648 (4) | −50.3 % | −50.4 % |
| wifi_tcp_up_mbps | 174.599 ± 8.499 (8) | 176.205 ± 9.077 (4) | +0.9 % | +0.5 % |
| wifi_tcp_down_mbps | 125.083 ± 13.26 (8) | 120.741 ± 17.762 (4) | −3.5 % | +2.8 % |

saved /home/kshitij/bwtracker/experiments/results/overhead_20261031-005043.json
```

Fig. 13. Throughput with vs without the tracker, pinned cores, OFF-ON-OFF (Table VII).

```
kshitij@kasodkar:~/bwtracker $ cd ~/bwtracker
kshitij@kasodkar:~/bwtracker $ sudo python3 experiments/proof/diag_processes.py
[sudo] password for kshitij:
round 1: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 2: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 3: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 4: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 5: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
kprobe misses 0 perf lost 0
saved /home/kshitij/bwtracker/experiments/results/diag_processes_20261031-003010.json
```

Fig. 14. Follow-up of the 49/50 result: 5 rounds × 50 processes, all exact.

```
bwtrack - eBPF per-process bandwidth   00:39:47   interval 1.00s
Apps (socket layer)  up     1.8 kbit/s  down     7.6 Mbit/s   | TCP wire up    1.8 kbit/s  retrans 0.00%  UDP drops 0
NIC (ground truth)   wlan0: up 169.3 kbit/s down 8.0 Mbit/s
tracker cpu 0.0%  sockets 6  kprobe misses 0  perf lost 0
view: PROCESSES   proto: ALL   sort: total rate   loopback: hidden (0)   idle: hidden
[p]rocesses [f]lows [t]proto [s]ort [l]oopback [i]dle [e]vents [q]uit

   PID PROCESS         PROTO   REMOTES   UP app/s  DOWN app/s  UP wire/s  RETX%     RTT RETX segs UDP drop  TOTAL UP
 24307 curl            TCP/UDP       2       0 B/s   926.4 KB/s       0 B/s      -   63.4ms         -        -    1.8 KB
  1277 sshd-session    TCP           1     220 B/s       0 B/s     220 B/s   0.0%    8.5ms         2        -   15.1 KB
recent events (retransmissions = packet loss on the path) ------------------------------------------------------------------
00:39:30 RETRANS  sshd-session(1277)     TCP 10.58.105.107:22 -> 10.58.105.150:4700  retransmitted 84 B
00:39:30 RETRANS  sshd-session(1277)     TCP 10.58.105.107:22 -> 10.58.105.150:4700  retransmitted 132 B
```

Fig. 15. `bwtrack live` during a 50 MB download: per-process rates, NIC counters and live retransmission events.

```
kshitij@kasodkar:~/bwtracker $ sudo ./bwtrack report --since 2h
Top programs (2026-10-30 22:41 -> now), loopback excluded
PROGRAM          PIDS        SENT    RECEIVED       TOTAL  TCP RETX UDP DROP  REMOTE IPS
curl             1          1.8 KB    47.8 MB    47.8 MB     0.00%        -  2606:4700:7::da, 10.58.105.124
sshd-session     3         29.8 KB     643 B     30.5 KB     0.74%        -  10.58.105.150
sshd-auth        1          2.9 KB     3.6 KB     6.5 KB     0.00%        -  10.58.105.150
(shown rows total)         34.6 KB    47.8 MB    47.8 MB
```

Fig. 16. `bwtrack report` from the SQLite history after the download.

```
Trial compile + attach of the real eBPF program...
    Compiling eBPF program against the running kernel's headers (BCC/clang)...
      BCC's bpf.h is older than this kernel's headers; adding 3 definition(s) from the kernel
    Compiled in 9.3s. Attaching probes...
    Attached 13 probes, skipped 2 optional
    attached kprobe     tcp_sendmsg
    attached kretprobe  tcp_sendmsg
    attached kprobe     tcp_cleanup_rbuf
    attached kprobe     tcp_rcv_established
    attached kprobe     udp_sendmsg
    attached kretprobe  udp_sendmsg
    attached kprobe     udpv6_sendmsg
    attached kretprobe  udpv6_sendmsg
    attached kprobe     skb_consume_udp
    attached kprobe     __udp_enqueue_schedule_skb
    attached kretprobe  __udp_enqueue_schedule_skb
    attached tracepoint tcp:tcp_retransmit_skb
    attached tracepoint sock:inet_sock_set_state
    skipped  kprobe     tcp_sendpage: not in this kernel
    skipped  kretprobe  tcp_sendpage: not in this kernel
[ OK ] eBPF program loads and attaches

All checks passed. Start with:  sudo ./bwtrack live
```

Fig. 17. `bwtrack doctor --compile` on kernel 6.18: three missing BCC definitions added automatically, 13 probes attached.

```
00:02:58 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 9000312984192246953, 'rate': 20000000.0, 'seconds': 5.0, 'size': 12
00}
session error: peer closed
00:03:13 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:03:16 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:03:18 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 7857147549393600850, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:03:26 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 7524465159335834034, 'rate': 20000000.0, 'seconds': 5.0, 'size': 12
00}
00:03:34 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:03:36 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:03:49 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 7676203071384277311, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:03:56 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 13950300188404019233, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:04:04 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:04:07 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:04:20 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 14183108569582350877, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:04:27 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 15207383299027626088, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:04:35 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:04:53 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:05:10 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 16048620837773356730, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:05:17 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 12045118306461216257, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:05:25 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:07:42 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:08:10 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 12530196772332802896, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
```

Fig. 18. The peer laptop running `trafficgen.py server`: one line per trial requested by the Pi.